

Java Date and Time:

```
import java.io.*;

import java.math.*;

import java.security.*;

import java.text.*;

import java.util.*;

import java.util.concurrent.*;

import java.util.function.*;

import java.util.regex.*;

import java.util.stream.*;

import static java.util.stream.Collectors.joining;

import static java.util.stream.Collectors.toList;

class Result {

    public static String findDay(int month, int day, int year) {

        Calendar cal = Calendar.getInstance();

        cal.set(year, month - 1, day);

        String[] days = {

            "SUNDAY",

            "MONDAY",

            "TUESDAY",

            "WEDNESDAY",

            "THURSDAY",

            "FRIDAY",

            "SATURDAY"

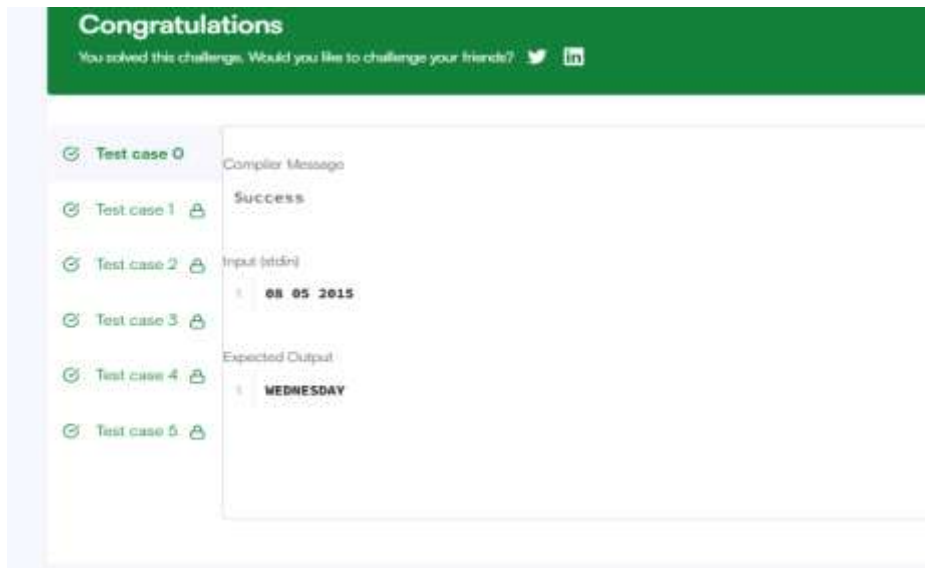
        };

        return days[cal.get(Calendar.DAY_OF_WEEK) - 1];

    }

}
```

```
public class Solution {  
    public static void main(String[] args) throws IOException {  
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));  
        BufferedWriter bufferedWriter = new BufferedWriter(new  
        FileWriter(System.getenv("OUTPUT_PATH")));  
  
        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");  
  
        int month = Integer.parseInt(firstMultipleInput[0]);  
  
        int day = Integer.parseInt(firstMultipleInput[1]);  
  
        int year = Integer.parseInt(firstMultipleInput[2]);  
  
        String res = Result.findDay(month, day, year);  
  
        bufferedWriter.write(res);  
        bufferedWriter.newLine();  
  
        bufferedReader.close();  
        bufferedWriter.close();  
    }  
}
```



2. Number of Days Between Two Dates

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
class Solution {
    public int daysBetweenDates(String date1, String date2) {
        LocalDate d1 = LocalDate.parse(date1);
        LocalDate d2 = LocalDate.parse(date2);

        return Math.abs((int) ChronoUnit.DAYS.between(d1, d2));
    }
}
```



3. Day of the Year

```
class Solution {
```

```
public int dayOfYear(String date) {  
    String[] parts = date.split("-");  
    int year = Integer.parseInt(parts[0]);  
    int month = Integer.parseInt(parts[1]);  
    int day = Integer.parseInt(parts[2]);  
  
    int[] daysInMonth = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};  
  
    // Check if the year is a leap year  
    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {  
        daysInMonth[1] = 29;  
    }  
  
    int dayCount = day;  
    for (int i = 0; i < month - 1; i++) {  
        dayCount += daysInMonth[i];  
    }  
  
    return dayCount;  
}
```

Input

```
date =  
"2019-01-09"
```

Output

```
9
```

Expected

```
9
```

4. Day of the Week

```
import java.time.LocalDate;

import java.time.format.TextStyle;

import java.util.Locale;

class Solution {

    public String dayOfTheWeek(int day, int month, int year) {

        LocalDate date = LocalDate.of(year, month, day);

        return date.getDayOfWeek()

            .getDisplayName(TextStyle.FULL, Locale.ENGLISH);

    }

}
```

✓ Case 1

✓ Case 2

✓ Case 3

Input

day =
31

month =
8

year =
2019

Output

"Saturday"

Expected

"Saturday"

5. Java Priority Queue

```
import java.util.ArrayList;

import java.util.List;
```

```
import java.util.Scanner;
import java.util.PriorityQueue;
import java.util.Comparator;
class Student {
    private int id;
    private String name;
    private double cgpa;

    public Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }

    public int getID() {
        return id;
    }

    public String getName() {
        return name;
    }

    public double getCGPA() {
        return cgpa;
    }
}

class Priorities {

    public List<Student> getStudents(List<String> events) {
```

```

PriorityQueue<Student> pq = new PriorityQueue<>(new Comparator<Student>() {

    @Override

    public int compare(Student s1, Student s2) {

        if (Double.compare(s2.getCGPA(), s1.getCGPA()) != 0) {
            return Double.compare(s2.getCGPA(), s1.getCGPA());
        }

        if (!s1.getName().equals(s2.getName())) {
            return s1.getName().compareTo(s2.getName());
        }

        return Integer.compare(s1.getID(), s2.getID());
    }
});

for (String event : events) {
    String[] arr = event.split(" ");

    if (arr[0].equals("ENTER")) {
        pq.add(new Student(
            Integer.parseInt(arr[3]),
            arr[1],
            Double.parseDouble(arr[2])
        ));
    } else if (arr[0].equals("SERVED")) {
        if (!pq.isEmpty()) {
            pq.poll();
        }
    }
}
}

```

```

List<Student> result = new ArrayList<>();

while (!pq.isEmpty()) {
    result.add(pq.poll());
}

return result;
}
}

public class Solution {
    private final static Scanner scan = new Scanner(System.in);
    private final static Priorities priorities = new Priorities();

    public static void main(String[] args) {
        int totalEvents = Integer.parseInt(scan.nextLine());
        List<String> events = new ArrayList<>();

        while (totalEvents-- != 0) {
            String event = scan.nextLine();
            events.add(event);
        }

        List<Student> students = priorities.getStudents(events);

        if (students.isEmpty()) {
            System.out.println("EMPTY");
        } else {
            for (Student st: students) {
                System.out.println(st.getName());
            }
        }
    }
}

```

```

    }
}
}
}

```



6. Java ArrayList

```

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

```

```

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int n = scanner.nextInt();
        List<List<Integer>> lines = new ArrayList<>();

        // Read all n lines of numbers
        for (int i = 0; i < n; i++) {
            int d = scanner.nextInt(); // Number of elements in this line
            List<Integer> line = new ArrayList<>();
            for (int j = 0; j < d; j++) {

```

```

        line.add(scanner.nextInt());
    }
    lines.add(line);
}

int q = scanner.nextInt(); // Number of queries

// Process each query
for (int i = 0; i < q; i++) {
    int x = scanner.nextInt(); // Line index (1-based)
    int y = scanner.nextInt(); // Position index (1-based)

    // Convert 1-based indexing to 0-based indexing
    int lineIndex = x - 1;
    int posIndex = y - 1;

    // Check if line index and position index are within valid bounds
    if (lineIndex >= 0 && lineIndex < lines.size()) {
        List<Integer> targetLine = lines.get(lineIndex);
        if (posIndex >= 0 && posIndex < targetLine.size()) {
            System.out.println(targetLine.get(posIndex));
            continue;
        }
    }

    System.out.println("ERROR!");
}

scanner.close();
}
}

```

The screenshot shows a coding platform interface with a sidebar on the left containing a list of test cases from 'Test case 0' to 'Test case 5'. The main area displays the input for 'Test case 0' as a 3x5 grid of numbers:

7	5			
8	1	3		
9	3	4		
10	3	1		
11	4	3		
12	5	5		

Below the input grid, the 'Expected Output' is shown as a 4x1 column of values:

1	74
2	52
3	37
4	ERROR!
5	ERROR!

7. Largest Number

```
import java.util.Arrays;
```

```
class Solution {
```

```
    public String largestNumber(int[] nums) {
```

```
        // Convert integer array to String array
```

```
        String[] strNums = new String[nums.length];
```

```
        for (int i = 0; i < nums.length; i++) {
```

```
            strNums[i] = String.valueOf(nums[i]);
```

```
        }
```

```
        // Custom comparator: compare concatenated strings (b + a vs a + b)
```

```
        Arrays.sort(strNums, (a, b) -> (b + a).compareTo(a + b));
```

```
        // Edge case: if the largest number after sorting is "0", the entire result is "0"
```

```
        if (strNums[0].equals("0")) {
```

```
            return "0";
```

```
        }
```

```

        // Build the final result

        StringBuilder result = new StringBuilder();

        for (String num : strNums) {
            result.append(num);
        }

        return result.toString();
    }
}

```

Input

```

nums =
[10,2]

```

Output

```

"210"

```

Expected

```

"210"

```

8. Java Comparator:

```

import java.util.*;

class Checker implements Comparator<Player> {

    @Override

    public int compare(Player a, Player b) {

        // If scores are equal, sort alphabetically by name
        if (a.score == b.score) {
            return a.name.compareTo(b.name);
        }

        // Otherwise, sort in descending order of score
        return b.score - a.score;
    }
}

```

```
}
```

```
class Player{
```

```
    String name;
```

```
    int score;
```

```
    Player(String name, int score){
```

```
        this.name = name;
```

```
        this.score = score;
```

```
    }
```

```
}
```

```
class Solution {
```

```
    public static void main(String[] args) {
```

```
        Scanner scan = new Scanner(System.in);
```

```
        int n = scan.nextInt();
```

```
        Player[] player = new Player[n];
```

```
        Checker checker = new Checker();
```

```
        for(int i = 0; i < n; i++){
```

```
            player[i] = new Player(scan.next(), scan.nextInt());
```

```
        }
```

```
        scan.close();
```

```
        Arrays.sort(player, checker);
```

```
        for(int i = 0; i < player.length; i++){
```

```
            System.out.printf("%s %s\n", player[i].name, player[i].score);
```

```
        }
```

```
}
```

}

The screenshot shows a coding platform interface. On the left, there is a list of test cases, each with a green checkmark icon and a lock icon. The test cases are labeled 'Test case 0' through 'Test case 5'. To the right of the test cases, there is a table with two columns: an index and a string. The table contains the following data:

Index	String
1	5
2	amy 100
3	david 100
4	heraldo 50
5	aakashsha 75
6	aleksa 150

Below the table, there is a section labeled 'Expected Output' with a list of strings corresponding to the sorted order of the input:

Index	String
1	aleksa 150
2	amy 100
3	david 100
4	aakashsha 75
5	heraldo 50

9. Sort the People

```
import java.util.Arrays;
```

```
class Solution {  
    public String[] sortPeople(String[] names, int[] heights) {  
        int n = names.length;  
        Person[] people = new Person[n];  
  
        for (int i = 0; i < n; i++) {  
            people[i] = new Person(names[i], heights[i]);  
        }  
  
        // Sort descending by height  
        Arrays.sort(people, (a, b) -> Integer.compare(b.height, a.height));  
  
        String[] result = new String[n];  
        for (int i = 0; i < n; i++) {  
            result[i] = people[i].name;  
        }  
    }  
}
```

```
        return result;
    }

    private static class Person {

        String name;

        int height;

        Person(String name, int height) {

            this.name = name;

            this.height = height;

        }

    }

}
```

Input

```
names =
["Mary", "John", "Emma"]
```

```
heights =
[180, 165, 170]
```

Output

```
["Mary", "Emma", "John"]
```

Expected

```
["Mary", "Emma", "John"]
```